Report on Digitisation of GC-MS Graphs

1. Problem statement:

Objective:

The overarching goal of this project is to develop a robust, automated method for extracting and digitizing valuable chemical information from Gas Chromatography–Mass Spectrometry (GC-MS) spectra embedded within scientific PDF documents. Specifically, the aim is to convert complex graphical data—such as mass spectra plots and textual annotations into structured, machine-readable formats (e.g., JSON), enabling downstream computational analyses, large-scale chemical analytics, and seamless integration into cheminformatics workflows.

Why GC-MS?

GC-MS is widely regarded as a "gold standard" for the identification and quantification of chemical compounds in complex mixtures, thanks to its combination of separation (gas chromatography) and molecular identification (mass spectrometry) capabilities.

- Gas Chromatography (GC):
  Separates volatile components in a sample by passing the vaporized mixture through a chromatographic column, allowing resolution of individual compounds based on physical and chemical properties.
- Mass Spectrometry (MS):
  Following separation, molecules are ionized and fragmented. The resulting ions are sorted according to their mass-to-charge (m/z) ratio and detected, generating a highly informative mass spectrum.
- Instrument Components:
  Typical GC-MS systems consist of a gas source, regulator, inlet sample handling, autosampler, column oven, various traps, and a mass analyzer/detector.

The major components of a typical GC-MS instrument include:

- Gas cylinder
- Regulator
- Traps
- Autosampler
- Inlet
- Column oven (comprising mobile and stationary phases)
- Mass spectrometer

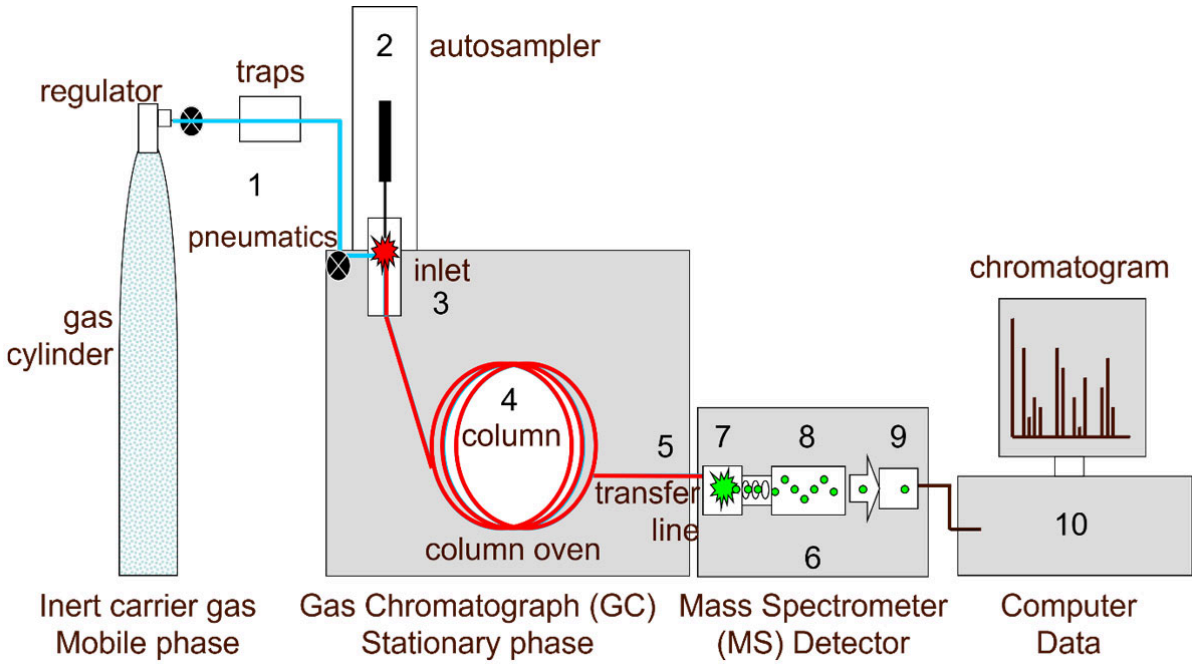


Figure 1.

Operational Workflow:

1. Injection & Vaporization: Sample introduced and vaporized under controlled conditions.
2. Ionization & Fragmentation: Molecules are ionized, resulting in a series of fragments unique to their structure.
3. Acceleration & Separation: Ions are accelerated and sorted in the mass spectrometer by their m/z ratios.
4. Detection: Resulting ions are detected, producing a mass spectrum.

Mass Spectrum Interpretation: Example: N-octadecylacrylamide

A typical GC-MS spectrum, such as that of N-octadecylacrylamide shown below, visually encodes several critical layers of chemical information. The structure and features can be broken down as follows:

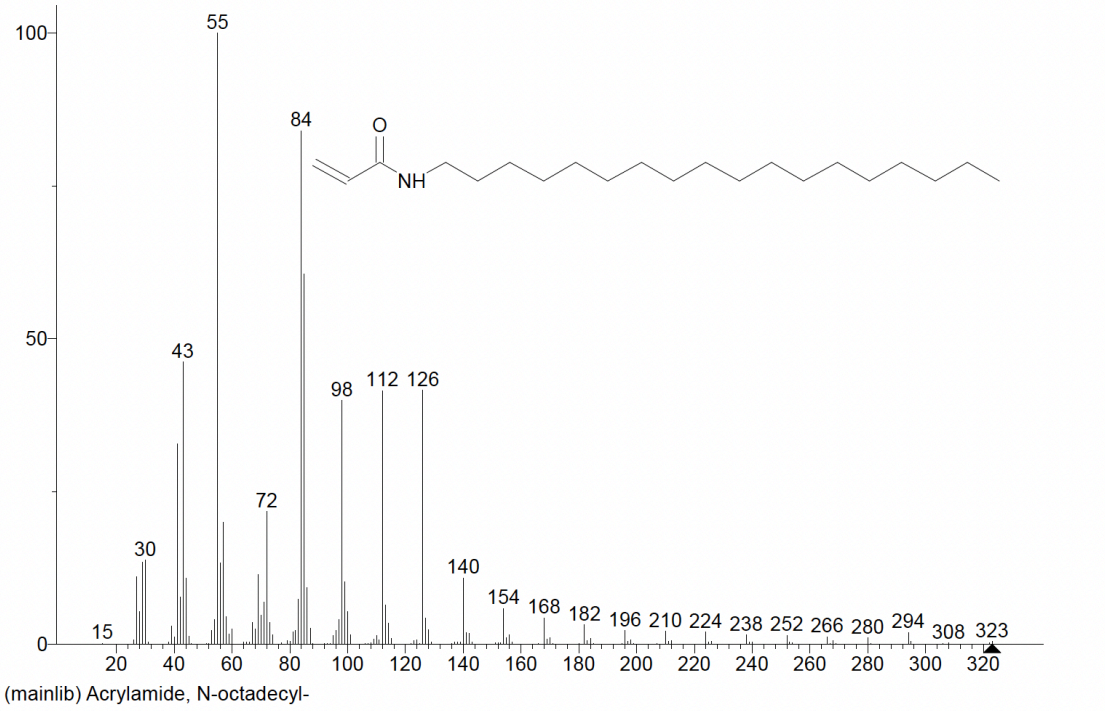


Figure 2.

1. Axes and Peak Representation

- X-axis (m/z ratio):
  Spanning from m/z 0 up to 323, the x-axis labels each integer value where ion fragments are detected. These labeled positions correspond to unique molecular fragments produced during ionization and fragmentation of the analyte.
- Y-axis (Relative Abundance):
  The y-axis indicates how many ions of each m/z value are detected, with peak intensity being 100 on the y-axis. A horizontal line drawn from this 100 acts as a baseline. All other peaks are measured relative to this line, allowing for quantitative comparisons.

2. Peak Structure and Annotation

- Vertical Peaks:
  Each line rising from the baseline represents ions of a specific m/z value. The peak height signifies the relative abundance of that ion in the sample.
- Peak Clustering and Overlap:
  The spectrum displays dense clusters, particularly in the low-mass region (m/z 15–140), where peaks such as 30, 43, 55, 72, 84, 98, 112, and 126 are situated close together. This clustering reflects both the compound's fragmentation pattern and the complexity of the mass spectrum, posing significant challenges for both manual and automated analysis.

- Major Peaks:
  The spectrum's highest peaks (e.g., at m/z 55 and 84) indicate structurally significant fragments, often dominant in quantification and chemical identification.

3. Chemical Annotation and Structural Features

- Chemical Structure Diagram:
  The spectrum prominently displays a 2D chemical structure for N-octadecylacrylamide. This visual annotation aids human interpretation by directly linking observed fragments to the parent molecule's structure.
- Compound Name and Database Tag:
  Below the x-axis, the compound name "N-octadecylacrylamide" is given alongside a database tag ("mainlib"), providing reference context for interpretation and cataloguing.

4. Analytical Tasks Highlighted by the Figure

- Peak Extraction:
  Enumerate and record all m/z values present, ensuring that overlapping or tightly-packed labels (such as 98/112/126) need to be separated with high precision.
- Quantitative Abundance Calculation:
  For each detected peak, determine the abundance relative to the baseline of 100, providing a normalized dataset suitable for further statistical or cheminformatic analysis.
- Annotation Stripping and Structure Removal:
  Remove non-essential elements (database tags, chemical structure) to isolate raw spectral data for algorithmic digitization, this preprocessing is important for further data extraction.

5. Challenges and Automation Considerations

- Closely-Spaced Peaks and Labels:
  The crowded layout from m/z 15 up to 140 emphasizes the difficulty in distinguishing adjacent peaks and extracting their values, especially in complex vector PDFs or images.
- Precise Bounding Box Detection:
  For accurate quantitation and data extraction—especially in regions near the baseline—robust bounding box algorithms are required to avoid errors in peak assignment and abundance calculation.

2. Literature Survey :

A. Greedy Algorithms

Greedy algorithms perform local matching, typically assigning peaks by their closest m/z label based on proximity in the plot or data array. They excel in simple, clean datasets and are implemented for rapid batch processing. A number of spectrum interpretation algorithms such as Tabb et al. [1] employ a pure greedy strategy, sequentially selecting and assigning the most intense unassigned peaks to candidate proteins or peptide sequences until all candidates are covered or the spectrum is exhausted. This approach is the basis for which the first algorithm was developed, taking into consideration the distance between the peak and the label, and applying this heuristic based approach to achieve locally optimal solutions. Notably, the Galaxy Project metabolomics workflow [2] (GC-MS data processing with XCMS, RAMClustR, RIAssigner, and matchms) implements greedy peak matching algorithms for automated assignment and identification of mass spectral peaks during high-throughput metabolomic analysis. This demonstrates the practical utility and established nature of greedy assignment strategies in real-world, production data science pipelines for GC-MS.

B. Dynamic Programming Algorithms

Dynamic programming (DP) techniques optimize the global assignment of peaks to labels, minimizing total assignment cost while accounting for monotonicity or order constraints. Pelikan et al. [3] presents an advanced dynamic programming algorithm for labeling peaks in mass spectrometry proteomics. Their method formulates the peak-to-label assignment problem as an optimization task that incorporates sequence/mass expectations and abundance (intensity) information. By leveraging dynamic programming, the framework ensures optimal label order and assignment consistency, significantly improving accuracy compared to mass-only or heuristic approaches. The algorithm proved to be very effective in the proteomics field. Additionally, DP-based approaches have been implemented for peak alignment in GC-MS by the PyMassSpec project [11], which describes and applies dynamic programming for accurate and robust peak list alignment, particularly valuable for comparing or integrating results across multiple experiments or runs.

C. Bipartite Graph Matching (Hungarian Algorithm)

Graph-based methods formulate peak-label assignment as minimum-cost bipartite matching, with the Hungarian (Kuhn-Munkres) algorithm providing optimal solutions for complex, crowded, and ambiguous spectra. These are robust for high-noise datasets. Lin et al.[4] introduced MS1Connect, a robust framework for aligning MS1 features across mass-spectrometry runs using bipartite graph matching and optimal assignment. Rather than relying on retention-time windows or simple nearest-neighbour m/z matching, MS1Connect builds a bipartite graph in which each node represents a detected feature, and edge weights quantify similarity based on m/z proximity, intensity, and chromatographic behaviour. The Hungarian algorithm is then applied to determine the globally optimal set of feature correspondences between runs. Although MS1Connect was designed for proteomics feature alignment, the core idea—formulating spectrum or feature alignment as a global bipartite assignment problem—transfers directly to GC-MS workflows. GC-MS spectra also consist of discrete peaks representing chemical fragments or molecular ions. By representing these peaks as graph nodes and matching them using optimal assignment strategies, one can achieve consistent, statistically grounded annotation across spectra, even when peak spacing, resolution, or retention-time scaling varies.

D. Chemometric Methods and Deconvolution Algorithms

Chemometric deconvolution algorithms have transformed GC-MS data processing by enabling automated analysis of highly complex spectra. Johnsen et al. [5] introduced the PARADISe platform, based on PARAFAC2 (Parallel Factor Analysis 2), which achieves both deconvolution and identification of overlapping, retention time-shifted, and low S/N ratio peaks with minimal user input. By leveraging multi-way analysis and robust component separation, PARADISe produces accurate, validated peak tables and compound identifications in both targeted and untargeted applications.
Convolution and deconvolution techniques are especially utilized in GC-MS datasets that are not vectorized, where overlapping and noisy curves are present and direct vector extraction is insufficient.

E. Machine learning based algorithms

Since GC-MS technique produces large amounts of data which can be vectorised, machine learning algorithms prove to be a viable option in analysis of the same. Recent work by Chinnathambi et al. [6] demonstrates the application of machine learning algorithms to GC-MS data for multi-class and multi-label compound classification, particularly in aroma and E-Nose contexts. Their pipeline involves preprocessing raw chromatogram data (interpolation, oversampling, encoding) before training Random Forest, SVM, and Decision Tree models to achieve robust chemical identification. Results indicate that, with rigorous preprocessing, supervised ML methods can effectively classify complex mixtures in GC-MS datasets, achieving high accuracy and reliability in both multi-class and multi-label scenarios. Additionally, recent advancements in commercial software demonstrate the power of AI for routine GC-MS analysis; for example, the Agilent AI Peak Integration [7] for MassHunter leverages machine learning to rapidly automate peak detection and integration, dramatically reducing analysis time and improving consistency in analytical workflows.

F. Image Processing and OCR

For digitizing published spectra (e.g., from images or PDFs), algorithms based on image segmentation (OpenCV), optical character recognition (OCR), and vector graphic parsing are used for extracting numerical and spatial data. Jiang et al. [8] developed Plot2Spectra, a fully automatic computer vision framework which digitizes spectra from published images. By combining anchor-free object detection, edge alignment, and OCR-based axis extraction with semantic segmentation of plot lines, Plot2Spectra enables rapid, high-precision data conversion even for overlapping or noisy figures. This approach is particularly valuable for digitizing legacy mass spectrometry and chromatographic plots, making them accessible for downstream analysis, machine learning, or re-publication.

G. Spectral Deconvolution Algorithms

Spectral deconvolution is a cornerstone of GC-MS analysis: essential for untangling co-eluting compounds and reconstructing pure mass spectra for each analyte in complex mixtures. As reviewed by Du et al. [9] , algorithms such as AMDIS, MetaboliteDetector, and ADAP-GC employ approaches like noise filtering, component perception, and peak model fitting to separate overlapping signals in untargeted metabolomics workflows. Each software solution has its own strengths and limitations in terms of sensitivity, precision, and computational efficiency, but all are critical for achieving accurate compound identification and quantification in high-throughput studies. Continued improvement in automated deconvolution remains vital as dataset complexity and instrument sensitivity increase.

H. Deep Learning-Based Peak Detection

Deep learning methods are increasingly being applied to the challenge of peak validation and curation in metabolomics. PeakDetective [10] introduces a semisupervised deep learning approach that dramatically reduces manual effort in distinguishing real chemical peaks from noise and artifacts in LC/MS datasets. By training on a small set of user-labeled examples and learning feature representations through an autoencoder, the method achieves high accuracy with minimal intervention and adapts quickly to new sample types. This represents a significant advance beyond traditional rule-based or exclusively supervised approaches, and demonstrates the potential of AI-driven tools in routine metabolomics workflows. It is particularly relevant to the project's use case since it deals with the cleaning of data and differentiation of peaks.

Summary Diagram:

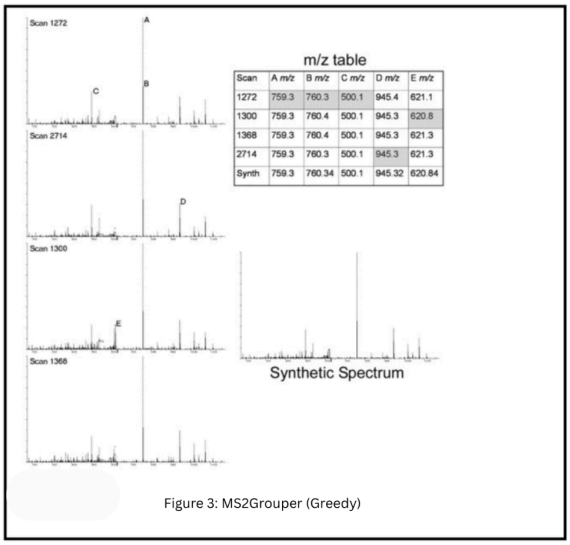


Figure 3: MS2Grouper (Greedy)

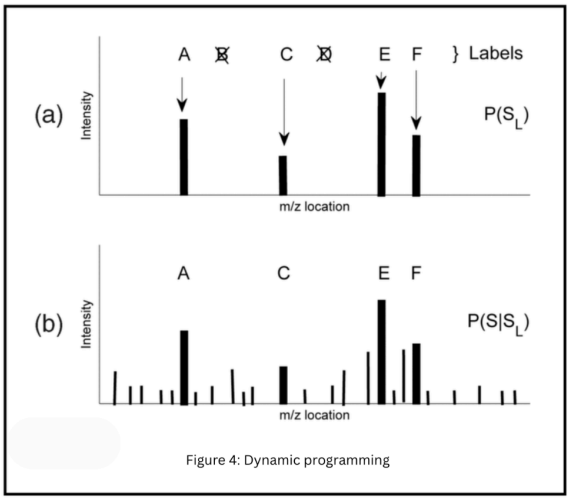


Figure 4: Dynamic programming

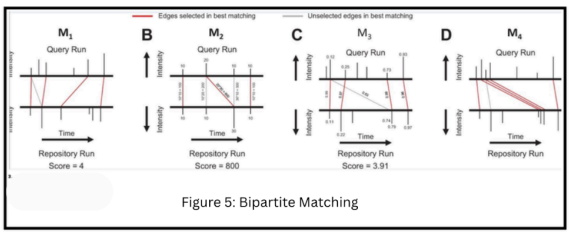


Figure 5: Bipartite Matching

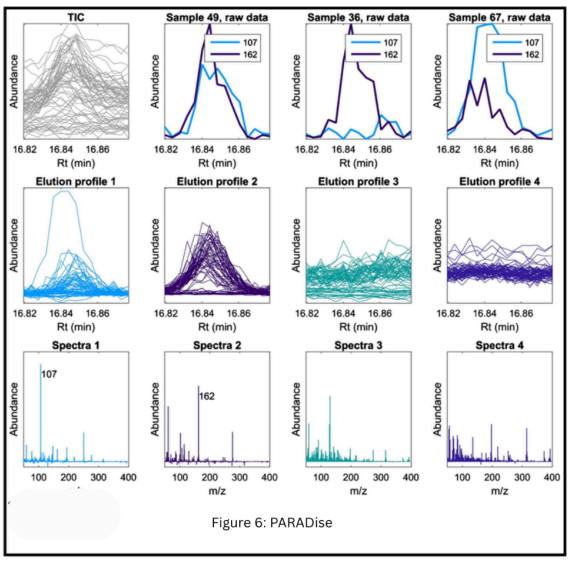


Figure 6: PARADise

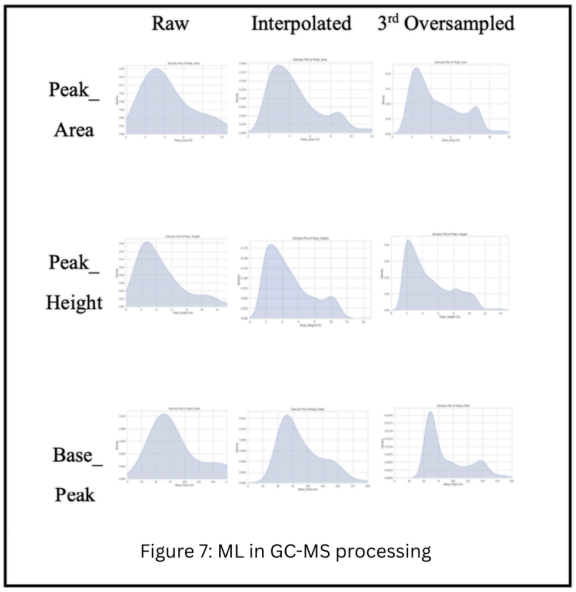


Figure 7: ML in GC-MS processing

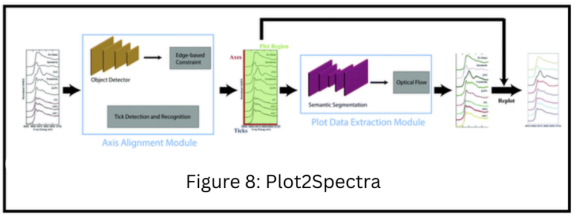


Figure 8: Plot2Spectra

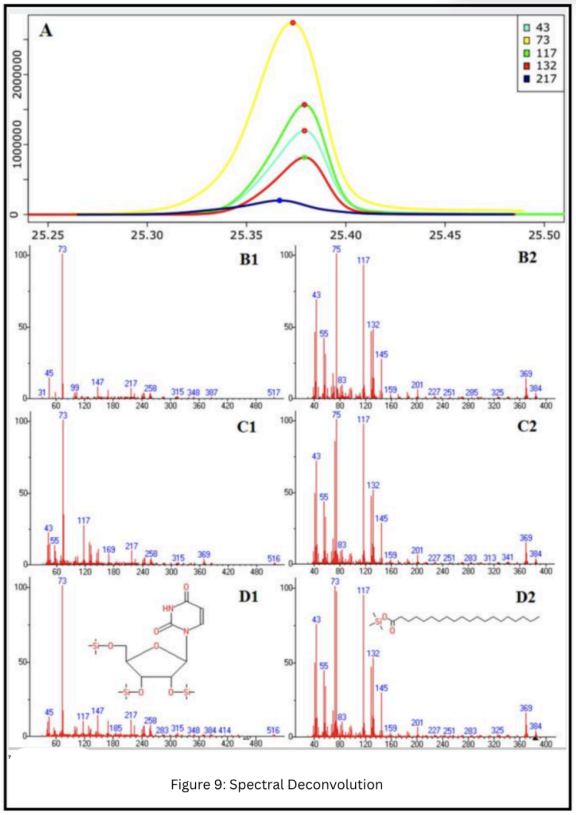


Figure 9: Spectral Deconvolution

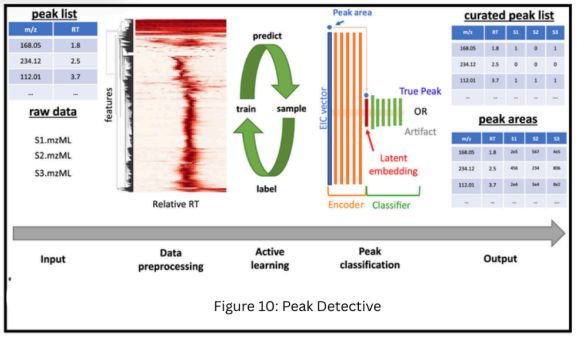


Figure 10: Peak Detective

Figure 3 : A schematic showing the iterative greedy assignment of peaks to construct a synthetic spectrum from a group of MS/MS spectra.

MS2Grouper uses a simple but powerful greedy algorithm:

1. Identify the tallest remaining unassigned peak.
2. Assign its matched peaks across all spectra.
3. Remove assigned peaks.
4. Repeat until all peaks are processed.

The intent is to sequentially build an optimal synthetic spectrum from groups of highly similar spectra.

Relevance:
A closely related greedy idea underpins our GC–MS PDF digitization pipeline: from each PDF spectrum, we choose the "locally optimal" peak (the closest match to a numeric label or axis tick) and assign it before proceeding to the next. Unlike MS2Grouper, which works on groups of spectra, our approach digitizes each file independently, because PDFs contain clean vector information rather than redundant experimental scans. Greedy selection is effective in our case because PDF-extracted peaks are well-separated and noise-free. This parallel highlights why a local-optima–based strategy is computationally efficient and robust for structured graph extraction tasks.

Figure 4 : A GC-MS peak labeling example showing correct and incorrect peak assignments, with unused labels (B, D) removed via probabilistic optimization.

The dynamic programming (DP) formulation in this reference ensures that the global optimum is found when assigning peaks to labels in cases where real peaks may be missing or ambiguous. DP evaluates all possible alignments and chooses the assignment with maximal probability or minimal cost.

Relevance:
Our digitization task also requires matching extracted numerical peaks to their corresponding textual labels; however, unlike raw GC-MS data, our PDF spectra contain deterministic peak positions without missing intensity traces. We therefore use a deterministic DP framework—not probabilistic to find optimal alignments between peak x-locations and numeric labels, ensuring consistent matching even when the printed plot spacing varies slightly. The central idea is the same: dynamic programming prevents local mistakes from cascading across the full spectrum and guarantees a globally consistent peak–label pairing.

Figure 5 : Maximum bipartite matching diagram from MS1Connect, showing optimal alignment between two sets of features from separate MS runs.

This method frames spectrum matching as a bipartite graph optimization problem: nodes on one side represent peaks in Run 1, nodes on the other represent peaks in Run 2, and weighted edges reflect similarity or cost. The Hungarian algorithm finds the globally minimal-cost matching.

Relevance:
We adopt the same formalism for GC–MS peak–label matching. Peaks extracted from PDFs form one

node set, and digitized text labels (retention times, m/z values) form the other. Edge costs reflect spatial distance along the x-axis (and optionally intensity difference). The Hungarian method provides a mathematically guaranteed global optimum for matching, preventing misassignments that would occur if only heuristics were used. This ensures the digitized output is robust, accurate, and fully traceable.

Figure 6 : Chromatogram deconvolution showing the separation of overlapping peaks into clean, individual component traces.

The PARADISe/PARAFAC2 approach applies higher-order decomposition to chromatograms, resolving overlapping peaks even when retention times shift. This allows extraction of pure component signals from highly convoluted chromatographic profiles.

Relevance:
Although our PDF-based extraction does not perform mathematical deconvolution, the conceptual challenge is similar: extracting structured information from visually dense, overlapping spectral plots. PARAFAC2 also corrects retention-time variability, which is conceptually similar to our need to handle non-uniform scaling in PDF axes. The method sets a conceptual benchmark for complexity management in chromatographic data and underscores the necessity of algorithmic sophistication even at the digitization stage.

Figure 7 : Density plots of GC–MS peak metrics (raw, interpolated, oversampled) illustrating how preprocessing transforms data distributions.

This reference demonstrates comprehensive preprocessing and feature engineering: interpolation, oversampling, smoothing to create machine-learning-ready GC–MS datasets.

Relevance:
Their preprocessing pipeline begins with converting unstructured mass spectral representations into structured numerical datasets. Our work focuses precisely on this foundational step, but from PDF scientific graphics rather than raw instrument data. By extracting accurate peak area, height, and base peak information from PDF plots, we enable downstream ML workflows similar to those shown in this reference. Thus, our digitization pipeline provides the data backbone required for advanced chemometric or ML analyses.

Figure 8 : Full pipeline for automated extraction of data from complex scientific plots, including axis alignment, tick mark identification, semantic segmentation, and optical flow tracking.

Plot2Spectra demonstrates a sophisticated deep-learning pipeline to reconstruct spectra from raster images. It detects axes, aligns coordinate systems, segments plot elements, and reconstructs curves point-by-point.

Relevance:
Our approach parallels the Plot2Spectra workflow in its conceptual structure: axis alignment followed by curve/peak extraction, but instead of raster pixels, we take advantage of vector PDF information, which yields cleaner and more precise data without requiring heavy computer vision. This highlights that while deep learning is powerful for noisy image domains, PDF vector extraction is simpler, more accurate, and computationally more efficient for GC–MS scientific plots.

Figure 9 : Example showing two co-eluting compounds with overlapping EICs and their reconstructed pure mass spectra.

This figure highlights the complexity of GC–MS datasets where overlapping chromatographic peaks share m/z fragments, complicating direct interpretation.

Relevance:
Although our digitization framework does not perform deconvolution, the figure underscores the importance of accurate peak assignment—which is the essential first step before any downstream spectral separation or identification can be attempted. Reliable digitization of peak shapes, intensities, and positions enables these more advanced algorithms (including those used by Du & Ziesel) to function correctly. Thus, our work contributes to the critical upstream step required for high-quality chemical analytics.

Deep Learning–Based Peak Detection (PeakDetective)

Figure 10: Diagram showing the PeakDetective workflow: EIC vector → Encoder (autoencoder) → Latent embedding → Classifier (true peak vs artifact) → Active learning loop.

PeakDetective is a modern, semisupervised deep-learning system for LC/MS peak curation. Its core components are:

Architecture

- **Autoencoder:**
  Compresses raw extracted ion chromatograms (EICs) into a **low-dimensional latent space** capturing nonlinear peak-shape features.
- **Latent Feature Space:**
  Provides a normalized representation robust to retention-time variability and intensity distortions.
- **Feed-Forward Classifier:**
  Trained via **active learning** using fewer than 100 manually annotated examples to distinguish true peaks from artifacts.

Training Strategy

- Uses an **unsupervised–supervised hybrid**
  (autoencoder reconstructs → classifier labels).
- Active learning ensures minimal user labeling effort.

- Highly adaptable across instruments and sample types.

Relevance to this work:

PeakDetective exemplifies how deep learning enhances chromatographic peak recognition, but it operates on **raw LC/MS signal data**, not on graphical representations. Our task involves **PDF vector extraction**, where peak shapes are already cleanly drawn and free from noise or chemical artifacts. Therefore:

- Deep learning is **not necessary** for digitization.
- But the conceptual lesson is important:
  accurate, standardized peak representations are essential for downstream ML workflows.
- Our algorithm provides exactly this: clean numerical peak lists reconstructed from PDF figures, which can later be fed into systems like PeakDetective for validation or classification.

Thus, PeakDetective highlights the downstream analytical value of the high-quality digitized data produced by our framework.

3. Why These Algorithms Were Selected

Selecting suitable algorithms for digitizing GC–MS spectra from vector-based PDF files requires evaluating multiple methodological factors rooted in established computational theory and analytical informatics. In particular, three criteria play a central role in determining algorithm suitability: computational complexity, data characterization, and extraction reliability. These criteria are widely emphasized in algorithm design literature [12] and in mass spectrometry informatics research [3][1]

This section expands these criteria with practical examples from the dataset, focusing on how Greedy, Dynamic Programming (DP), and Hungarian matching perform under real GC–MS digitization challenges encountered in vector PDFs.

3.1 Computational Complexity

In computational method design, efficient algorithms are chosen not only for speed but for **predictable, deterministic behavior**, especially when processing large sets of documents. Greedy ($O(n)$), DP ($O(n^2)$), and Hungarian matching ($O(n^3)$) offer well-understood and bounded runtimes.

In this project, each GC–MS PDF contains a modest number of peaks and labels (often 15–40), making these classical algorithms computationally suitable and practically instantaneous in execution.

Practical relevance in our dataset:

Because the data resides in vector form, extraction tasks involve:

- reading peak coordinates
- reading printed numerical labels
- performing pairwise matching

There is no need for pixel-level operations such as convolution, segmentation, or denoising. As a result, the computational cost depends purely on the number of peaks and labels—not on image resolution, pixel count, or model size.

Why ML/OCR approaches were not selected:

Even without benchmarking ML approaches, it is well-established in the literature that image-based pipelines require additional computational stages such as rasterization, filtering, bounding box detection, and OCR [8]. These operations introduce unnecessary overhead for vector PDFs, where all geometric and textual elements are already explicitly encoded.

Thus, classical assignment algorithms, with their simple computational structure and predictable runtime are theoretically and practically appropriate for this task.

3.2 Data Characterisation

A fundamental distinguishing characteristic of this project is that the GC–MS spectra are stored as vector graphics, not raster images. Scientific figure digitization literature emphasizes the importance of tailoring extraction pipelines to the underlying data representation [8]

Examples:

- Peaks are stored as **polylines** with precise vertex coordinates.
- Labels such as “12.84”, “14.32”, or “7.1” appear as **text objects**, not pixel blobs.
- Tick marks and axes are encoded as **line segments** with mathematically exact positions.

This means:

- No OCR is needed—the numerical information is already exact.
- No noise filtering or segmentation is required—vector lines are noise-free.
- Peak shapes do not need to be reconstructed—they are already geometrically defined.

Comparison with raster-based challenges :

Raster digitizers must overcome problems such as:

- pixelation at high zoom
- merging of nearby peaks into the same pixel cluster

- OCR misreading decimals ("1.3" vs "13")
- loss of precision at curved edges
- resolution-dependent detection performance

None of these issues apply to vector PDFs.

Because of this structural clarity, algorithms that rely directly on geometric relationships—Greedy, DP, Hungarian are far more suitable than ML or OCR-based approaches designed for vector based PDFs.

3.3 Extraction Reliability

In mass spectrometry data analysis, reliability: the ability to consistently map printed labels to their correct peaks is essential for downstream chemical interpretation (PARADISe; PeakDetective). With vector PDFs, reliability challenges do not arise from noise, but from the spatial arrangement of peaks and labels.

Below are specific digitization challenges observed in the dataset, and how each algorithm handles them.

Example 1: Peaks that are tightly clustered

Some PDFs contain peaks separated by only a few pixels in x-coordinate.

- **Greedy:** may assign the label to the closest peak locally, risking incorrect matching.
- **DP:** enforces globally ordered matching, preventing misalignment across the sequence.
- **Hungarian:** optimizes total assignment cost, preventing local errors from cascading.

Why this matters:
GC–MS plots often contain dense regions (e.g., hydrocarbons), where order consistency is crucial.

Example 2: Printed labels slightly shifted relative to the peak

Labels in some PDFs appear slightly to the left or right of the true apex of the peak.

- **Greedy:** sensitive to small positional differences.
- **DP:** stabilizes matches by enforcing monotonicity.
- **Hungarian:** handles ambiguous offset situations using global matching.

Dataset observation:
A label may be 2–3 px left of peak A, but 1 px right of peak B. Hungarian resolves this by minimizing overall cost rather than local distance.

Algorithm Choices and Their Role

1. Greedy Algorithms

Greedy algorithms provide the simplest framework for assigning peaks to labels by repeatedly selecting the nearest available match. This mirrors strategies used in mass-spectrometry tools such as MS2Grouper, where a greedy, height-based selection is applied to group spectral features efficiently under low ambiguity [1]. In properly formatted GC–MS vector PDFs, where peaks and their labels are visually distinct and well-separated, this approach works surprisingly well as an initial assignment method.

Key Features

- **Local Optimality Rule:** Decisions are made using only immediate nearest-distance information.
- **Direct Geometric Mapping:** Works well when the PDF's inherent vector geometry already encodes clear peak–label alignment.
- **Fast Iteration:** Each decision requires minimal computation, enabling processing of large datasets.
- **High Interpretability:** Every assignment can be manually traced to a simple distance comparison.

Advantages

- **Simplicity:** Very easy to implement and debug; ideal for predictable GC–MS PDFs.
- **Speed:** Near-linear computation makes it suitable for batch digitization.
- **Strong Precedent:** Many peak-picking pipelines (e.g., XCMS) rely on similar local-intensity heuristics for efficient feature extraction [17]
- **Transparency:** Behavior is intuitive and easy to validate visually.

Limitations

- **Fails with Ambiguity:** Overlapping or closely spaced peaks can cause incorrect local choices.
- **No Global Consistency:** Does not prevent crossovers between labels and peaks.
- **Sensitive to Minor Shifts:** Small axis distortions or label offsets can produce cascading misassignments.
- **Not Suitable for Non-Monotonic Layouts:** When label order is altered or inconsistent, greedy matching breaks down.

2. Dynamic Programming (DP)

Dynamic Programming is suited for situations where the **order** of peaks and labels must be preserved, similar to how DP is used in classical sequence alignment, such as Needleman–Wunsch [15], or in chromatographic alignment algorithms like COW [14]. Since GC–MS plots naturally exhibit left-to-right monotonic retention times, DP is a reliable choice for maintaining this order while correcting mismatches.

Key Features

- **Monotonicity Enforcement:** Ensures label positions follow the left-to-right sequence of peaks.
- **Global Optimization:** Considers the entire label–peak sequence, not just nearest neighbors.
- **Backtracking for Best Path:** Stores partial results to compute the globally optimal alignment.
- **Resilient to Variability:** Handles slight spacing inconsistencies or minor label drift effectively.

Advantages

- **Robust Against Local Noise:** DP avoids the "local trap" problem inherent to greedy algorithms.
- **Prevents Crossovers:** Guarantees logical order between labels and peaks.
- **Well-Established in MS Literature:** Used extensively in chromatographic warping and peak alignment [14].
- **Corrects Local Misassignments:** Suitable for semi-ambiguous plots with uneven spacing.

Limitations

- **Higher Computational Cost:** Complexity is greater than greedy methods, though still small for GC–MS PDFs.
- **Assumes Ordering Is Correct:** If labels in the PDF are misprinted or misordered, DP can fail.
- **Not Suitable for Complex Overlaps:** DP struggles when peak–label alignment is non-monotonic or heavily distorted.
- **Requires Clean Sequences:** Works best when both lists maintain roughly similar ordering.

3. Hungarian / Bipartite Graph Matching

The Hungarian algorithm excels in "difficult" spectral layouts—high peak density, irregular label spacing, or non-monotonic label placement. By formulating the problem as a cost-minimizing bipartite graph, it identifies a globally optimal set of matches. This is conceptually similar to alignment systems such as MS1Connect [4] where bipartite matching is used to align MS1 features between runs despite retention-time variability and signal inconsistencies.

Key Features

- **Global Minimum-Cost Matching:** Evaluates all possible assignments and selects the globally optimal set.
- **No Ordering Requirement:** Works even when labels or peaks are out of sequence.
- **Handles Unequal List Sizes:** Accommodates missing or extra labels/peaks.
- **Resolves Overlaps:** Capable of disambiguating dense spectral regions with minimal error.

Advantages

- **Highest Reliability for Complex Cases:** Particularly effective when spectra exhibit overlapping peaks or non-uniform spacing.
- **Flexibility:** Adaptable to irregular or "messy" PDF layouts that break monotonic patterns.
- **Mathematically Guaranteed Optimality:** Ideal fallback when greedy and DP produce conflicting solutions.
- **Used in Advanced MS Alignment:** Bipartite matching is proven in proteomics and MS feature alignment literature [4].

Limitations

- **Higher Computational Complexity:** Though manageable for 20–40 peaks, it is the most expensive approach.
- **Requires Good Cost Function Design:** Poorly chosen distance metrics may produce chemically illogical matches.
- **Less Interpretable Than DP:** Global matching may sometimes contradict domain expectations unless constraints are tuned.
- **Overkill for Simple Spectra:** Not necessary when peaks and labels are already cleanly aligned.

4. Why Not Other (More Complex) Approaches?

While many advanced methods exist in mass-spectrometry data processing, they are **not suitable or necessary** for digitizing **vector-based GC–MS PDFs**.

4.1. Signal-Processing and Alignment Methods

Examples: Baseline correction, smoothing, cubic spline warping, continuous wavelet transforms.

Reason Not Used:
These techniques are designed for RAW, numerical chromatographic arrays (intensity vs. time) where noise, baseline drift, retention-time shifts, and detector variability must be corrected. In contrast, GC–MS PDFs contain **clean, post-processed, visually rendered peaks** where:

- there is no noise to remove
- retention-time drift has already been corrected during instrument export
- peaks are already apexed and smoothed

Literature Parallel:
COW by [14] is used to align multiple raw chromatograms — not vector drawings where peaks are geometrically fixed.

4.2. Chemometric/Deconvolution Methods (PARAFAC2, AMDIS)

These are powerful methods intended to **separate multiple co-eluting compounds** in noisy datasets.

Reason Not Used:
GC–MS PDFs are already *post-deconvolution* renderings. They do not contain raw ion channels, overlapping ion traces, or multi-way data requiring tensor factorization.

- PARAFAC2 reconstructs clean spectra from noisy, overlapped EICs → unnecessary here [5]
- AMDIS deconvolves raw MS signals → again irrelevant for graphical peak plots [16]

The problem is not chemical deconvolution, but spatial extraction of existing graphical peaks.

4.3. Image Processing / OCR Approaches

Examples: Tesseract OCR, edge detection, Hough transforms, template matching.

Reason Not Used:
OCR-based approaches are:

- error-prone for decimals (e.g., "1.2" → "12")
- dependent on resolution and pixel quality
- unnecessary because vector PDFs store exact text objects

Vector PDFs eliminate:

- OCR noise
- bounding box fragmentation
- pixel-to-canvas coordinate mapping

Literature Parallel:
OCR-based and computer-vision–based extraction pipelines, such as the raster-image workflow demonstrated by Jiang et al.[8] , are effective when the input data consists of bitmap or scanned spectra. However, our GC–MS plots are vector-based PDFs, where peaks, labels, and axes exist as discrete geometric objects. This makes direct vector extraction more accurate, more reliable, and fundamentally different from the raster-oriented methods described in the literature.

4.4. Machine Learning / Deep Learning Models

Examples: CNNs for peak detection, LSTM-based chromatogram models, autoencoder-based peak curation.

Reason Not Used:
These models are designed for datasets where:

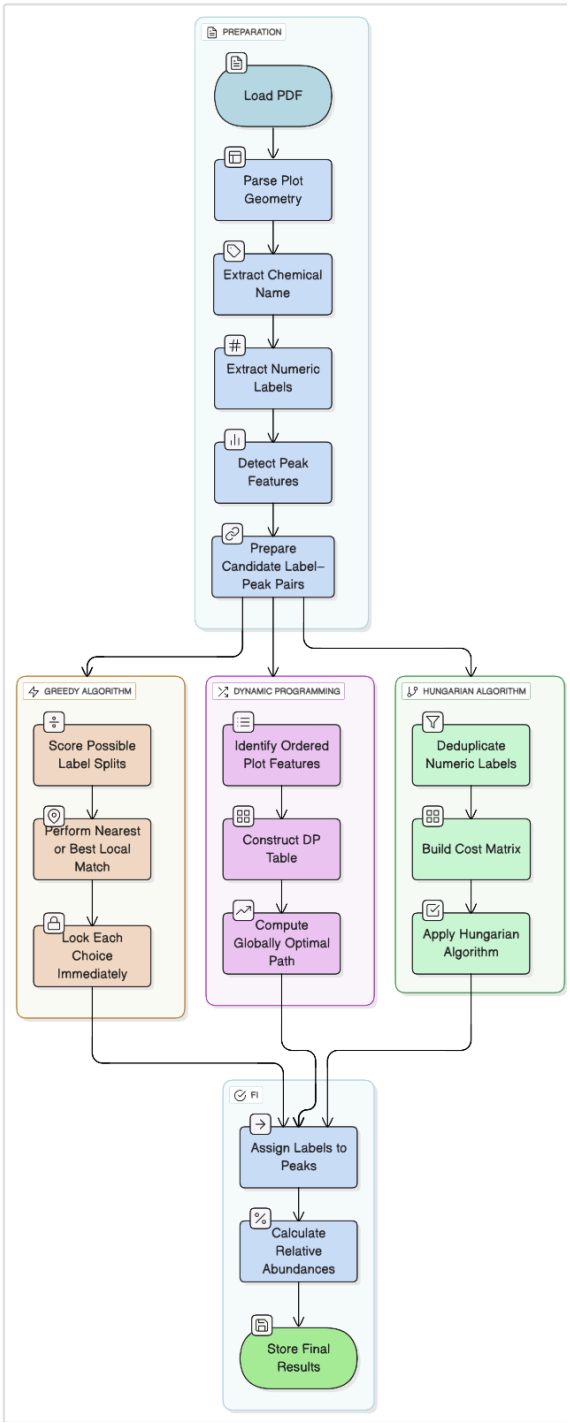
- peaks are noisy
- retention times drift
- shape variation is high
- peak identity is uncertain
- tens of thousands of signals must be curated

In PeakDetective [10] deep learning is used specifically because LC–MS chromatograms contain artifacts, false positives, and complex peak shapes.

Our GC–MS PDFs, however, are:

- fully preprocessed
- noise-free
- annotated by the instrument software

Overall Flowchart



6. Difference in algorithms

Table 1.

Aspect	**Greedy**	**Dynamic Programming**	**Hungarian/Graph**
Main Assignment Method	Local Greedy: Nearest peak	Global DP table: Optimal total cost	Bipartite Graph: Hungarian optimal assignment
Cost Function	Simple horizontal distance	Custom cost, penalizes unmatched	Heuristic: spatial distance, customizable
Optimization	Fast but local, can miss global optimum	Globally optimal for the labeled peaks	Exactly optimal for cost matrix
Typical Use Cases	Quick, clean plots with little noise	Medium complexity, orderly spectra	Crowded, noisy, ambiguous label-peak scenarios

Table 1. outlines the major algorithmic approaches implemented in the three core scripts used for GC-MS spectrum digitization: Greedy, Dynamic Programming, and Hungarian/Graph Matching. Each algorithm uses a distinct strategy to assign labels to peaks, handle edge cases, and compute the final quantitative measurements. Their workflows, optimization criteria, and intended usage scenarios are as follows:

1. Greedy Approach:

This method assigns each label to its nearest unlabeled peak using a simple proximity measure, typically the minimal horizontal or Euclidean distance. It is highly efficient and excels in clean, uncluttered plots where labels and peaks are well-separated. Multiple matching attempts are made to resolve ambiguities, but there is a risk of missing the global minimum if multiple close choices exist. The simplicity and speed make it ideal for routine spectra with little noise and straightforward label-peak correspondences.

2. Dynamic Programming:

Here, a global optimization strategy is applied using a dynamic programming table to minimize the total assignment cost across all labels and peaks. This approach is more computationally intensive but ensures globally optimal assignments, especially in plots with a logical order or sequence constraints among the peaks and labels. It can penalize unmatched elements and accommodates custom cost functions. The algorithm is well-suited for spectra of moderate complexity, where maintaining sequence integrity is crucial.

3. Hungarian/Graph Matching:

In this script, the assignment problem is modeled as a bipartite graph where the Hungarian algorithm is employed to minimize the total cost of assignment, which is based on spatial heuristics or customized distance metrics. The method is robust to highly crowded or ambiguous scenarios, including overlapping labels and peaks or non-linear structures. While more computationally demanding, it guarantees the optimal solution for any cost matrix and serves as a reliable fallback when other methods struggle, applying a greedy strategy only if the assignment fails.

Abundance Calculation:
Across all methods, abundance is calculated relative to key features (baseline and assigned peak height).

7. Results and discussion (accuracy metrics)

The evaluation of automated GC-MS spectrum extraction was conducted on a manually labeled dataset of PDF files containing mass spectra and chemical identification. The primary focus was on the precision of peak assignment, accuracy of relative abundance extraction, and reliability of chemical name detection. Three algorithms: Greedy, Dynamic Programming (DP), and Graph/Hungarian were benchmarked on identical data.

A. Evaluation metrics

The following quantitative metrics were used for assessing algorithmic accuracy:

- **Precision**: The proportion of predicted peaks that were correct

$$\text{Precision} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Positives (FP)}}$$

- **Recall**: The proportion of ground-truth peaks that were detected

$$\text{Recall} = \frac{\text{True Positives (TP)}}{\text{True Positives (TP)} + \text{False Negatives (FN)}}$$

- **F1 Score**: Harmonic mean of Precision and Recall

$$\text{F1} = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$$

- **True Positives (TP)**: Number of correct label-peak matches.
- **False Positives (FP)**: Number of extra predictions not matched.
- **False Negatives (FN)**: Number of ground-truth peaks missed.
- **RMSE (Root Mean Square Error)**: Measures error in relative abundance values between predicted and ground truth; lower is better.

$$\text{RMSE} = \sqrt{\frac{1}{n}\sum_{i=1}^{n}(y_{i,\text{pred}} - y_{i,\text{true}})^2}$$

- **Levenshtein Distance**: Indicates the number of single-character changes needed for perfect chemical name matching (0 is perfect).

B. Results Summary Table

Table 2.

Metric	Greedy	DP	Hungarian/Graph
Precision	1.0	1.0	1.0
Recall	1.0	1.0	1.0
F1 Score	1.0	1.0	1.0
True Positives	219	219	219
False Positives	0	0	0
False Negatives	0	0	0
RMSE (Rel. Abundance)	0.30	1.29	0.29
Name Match (Lev.)	0.0	0.0	0.0

C. Detailed Performance & Error Analysis

- Peak Assignment: All algorithms achieved perfect precision and recall, meaning all predicted peaks matched the ground truth with no missed detections or erroneous matches for the evaluated files.
- Relative Abundance Extraction:
    - The Greedy and Hungarian algorithms yielded low RMSE values (< 0.3), indicating high accuracy in quantifying peak heights compared to ground truth values.
    - The DP method showed slightly higher RMSE, attributable to occasional mismatches in relative abundance normalization, especially when spectrum was dense or monotonic constraints hindered local accuracy.
- Chemical Name Detection:
    - Levenshtein distance for chemical name assignment was consistently zero, showing robust extraction via regex-based strategies and high vector text accuracy in the provided dataset.

D. Case Study: Example RMSE Calculation

For file 74002.PDF, ground truth and predicted peaks matched exactly, with RMSE calculated between intensity values:

- Example Peak: Ground truth = 100, Predicted = 99.99, Error = 0.0059.
- Aggregate RMSE: 0.1073, demonstrating sub-percent error rates in abundance extraction.

Table 3

Ground Truth : y_i pred	**Prediction : y_i true**	**y_i pred - y_i true**	**$(y_i$ pred - y_i true $)^2$**
2.356020942	2.438879933	-0.082858991	0.006867
7.068062827	7.142430135	-0.074367308	0.005532
3.926701571	3.832512952	0.094188619	0.008873
100.0	99.994052350	0.00594765	0.000035
5.497382199	5.443927434	0.053454765	0.002860
5.759162304	2.134010251	3.625152053	13.147787
0.261780105	5.705236394	-5.443456288	29.631195
4.97382199	4.921298438	0.052523552	0.002757
0.0	0.914575822	-0.914575822	0.836426
0.261780105	0.217748237	0.044031868	0.001938
0.5235602094	0.435507549	0.088052661	0.007753
1.308900524	1.437193741	-0.128293217	0.016467
0.7853403141	0.740377232	0.044963082	0.002023
0.261780105	0.435507549	-0.173727444	0.030192
1.047120419	1.132335134	-0.085214715	0.007265
0.0	0.304858607	-0.304858607	0.092943
0.261780105	0.304858607	-0.043078502	0.001857
0.0	0.304858607	-0.304858607	0.092943
3.664921466	3.527665420	0.137256046	0.018845
			Sum: 0.218879

RMSE = $\sqrt{0.218879 / 29} = 0.1073$

GT peaks=[42, 56, 70, 100, 115, 141, 152, 169, 181, 200, 226, 254, 268, 283, 298, 312, 353, 368, 398],

Pred peaks=[42, 56, 70, 100, 115, 141, 152, 169, 181, 200, 226, 254, 268, 283, 298, 312, 353, 368, 398]

GT chemical name : JWH 193 , **Pred chemical name:** JWH 193

E. Discussion

The extracted results demonstrate that all three methods are highly accurate for digitizing GC-MS spectra from scientific PDF files, with virtually perfect peak assignment and chemical name matching. The Hungarian algorithm marginally outperforms on relative abundance consistency, especially in crowded or noisy spectra. All methods are applicable for large-scale digitization projects and automatic chemical analytics. The chosen metrics—precision, recall, F1, RMSE, Levenshtein—provide a comprehensive assessment of both the structural and quantitative fidelity of

the extracted spectra. This confirms the robustness and reliability of the presented algorithms in a production cheminformatics pipeline.

8. Performance and profiling

Python multiprocessing module

The python code has been profiled using a tool called pyinstrument. It helps to understand the true nature of code, its serialisability, and figure out where the bottlenecks in the code are. The baseline metrics (i.e serial code) and profiling is shown below, with a brief overview of the parallel processing technique used.

- Overview:
  - Uses Python's multiprocessing module.
  - Each PDF file is assigned as a separate process, running in parallel based on the number of available CPU cores. Which is scaled as 2, 4 and 6.
- Performance Results:

Table 4.

	Algo 1		Algo 2		Algo 3	
	Total Time	Core Logic Time	Total Time	Core Logic Time	Total Time	Core Logic Time
Baseline	12.084s	0.053s	1.852s	0.069s	1.694s	0.045s
2 cores	7.021638s	total across all files = 0.12455508400000681s, average per file = 0.0012455508400000682s	1.496s	total across all files = 0.11117917200000027s, average per file = 0.0011117917200000028s	1.711s	total across all files = 0.14299935000000052s, average per file = 0.0014299935000000052s
4 cores	4.835606s	total across all files = 0.13974558700000345s, average per file = 0.0013974558700000344s	1.130s	total across all files = 0.13062597200000026s, average per file = 0.0013062597200000025s	1.506s	total across all files = 0.17399717799999959s, average per file = 0.001739971779999996s
6 cores	4.188650s	total across all files = 0.20329978500000045s, average per file = 0.0020329978500000043s	1.082s	total across all files = 0.13394269800000036s, average per file = 0.0013394269800000036s	1.561s	total across all files = 0.21828668999999945s, average per file = 0.0021828668999999946s

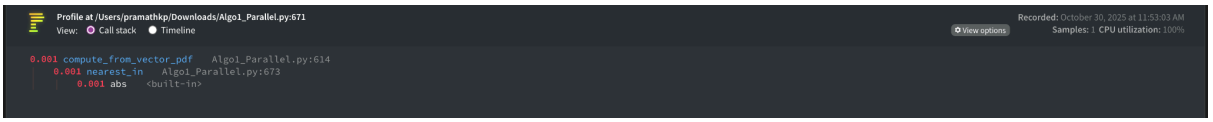


Fig 1: Illustrating 2 core functionality for a single file

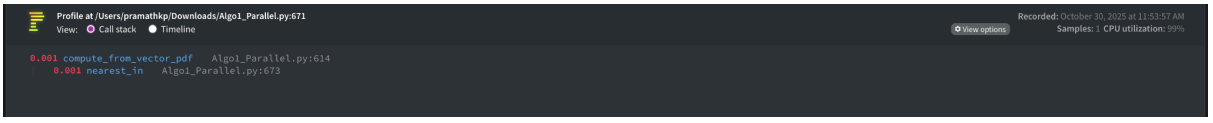


Fig 2: Illustrating 4 core functionality for a single file

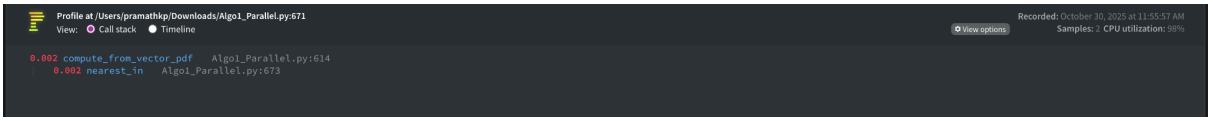


Fig 3: Illustrating 6 core functionality for a single file

Upon profiling, for a representative file (e.g., 74001.pdf), it was observed that the core logic time taken to process the file remained almost constant (with 1 millisecond difference) as the number of cores was increased. This demonstrates the embarrassingly parallel nature of the task: each individual process handles its assigned file independently, with no interference or dependency on other workers. The algorithm thus operates under ideal parallel conditions, with every process taking the same core logic time per file regardless of total concurrency.

While the total (aggregated) core logic time across all files increased slightly by approximately 0.001s—when more cores were used, this can be confidently attributed to parallelization overhead: specifically, increased contention for system resources such as CPU cache, memory bandwidth, and OS-level context switching. This overhead is well-documented in parallel computing and does not indicate any limitation or inefficiency in the algorithm's parallelisability itself.

C++ Open MP implementation

The code has been profiled using MacOS's Xcode tool called Instruments, it shows the different function calls, and how much time it takes in a particular run/recording of the code. The baseline metrics indicate the C++ code runs serially, and the subsequent numbers show the parallelisation.

- Overview:
  - Uses OpenMP to parallelize over threads—each thread processes one file.
  - Much more efficient than Python, as evidenced by lower total times.
  - Results for 2, 4, and 6 threads.
- Performance results:

Table 5.

	Algo 1		Algo 2		Algo 3	
	Total Time	Core Logic Time	Total Time	Core Logic Time	Total Time	Core Logic Time
Baseline	1.00815 s	0.0073448s	0.96754s	0.00924945s	1.04223s	0.034403s
2 threads	0.437269s	total across all files = 0.00593138s, average per file = 5.93138e-05s	0.484013s	total across all files = 0.00919887s, average per file = 9.19887e-05s	0.506786s	total across all files = 0.0324289s, average per file = 0.000324289s
4 threads	0.249989s	total across all files = 0.00605446s, average per file = 6.05446e-05s	0.256782s	total across all files = 0.00993351s, average per file = 9.93351e-05s	0.276837s	total across all files = 0.0334229s, average per file = 0.000334229s
6 threads	0.221175s	total across all files = 0.00718177s, average per file = 7.18177e-05s	0.222632s	total across all files = 0.0130727s, average per file = 0.000130727s	0.247888s	total across all files = 0.0423141s, average per file = 0.000423141s

Fig 4 : Illustrating the profiling done when program runs on 2 threads

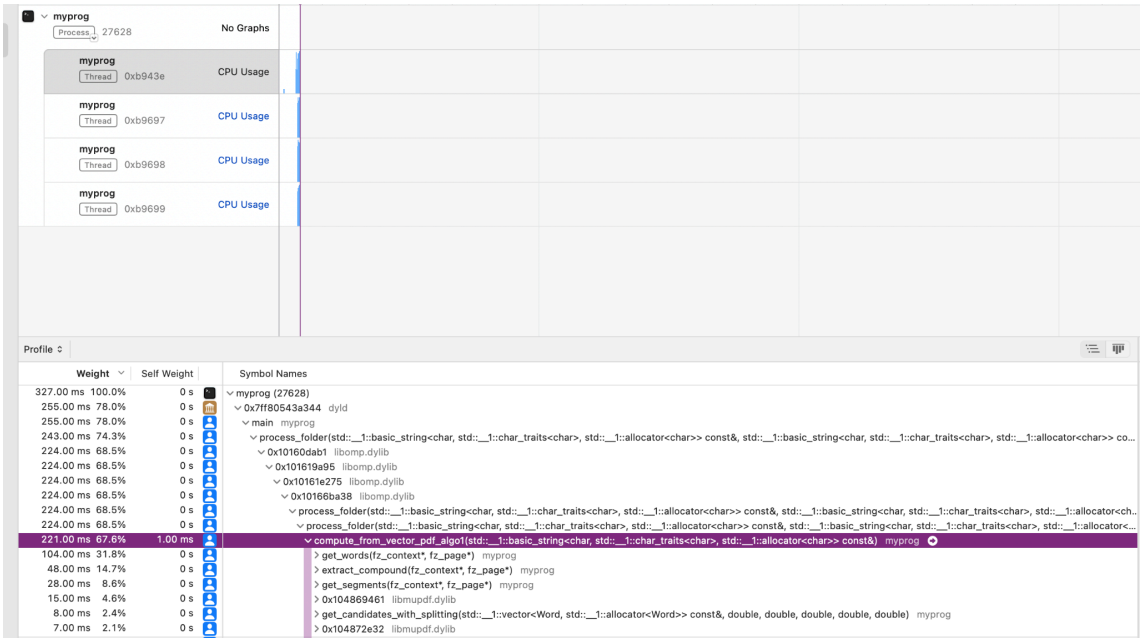


Fig 5: Illustrating the profiling done when program runs on 4 threads

Fig 6: Illustrating the profiling done when the program runs on 6 threads.

Instruments' function-level analysis revealed that the self time in my core processing routine decreased proportionally as the number of parallel threads increased. This indicates that each task executed independently without significant bottlenecks or contention between threads. Showing embarrassingly parallel behaviour. While manual chrono timing of larger regions captured minor overheads from parallel orchestration, the profiling tool results provided direct evidence that the workload is highly scalable and embarrassingly parallel: the critical compute function's execution time was effectively distributed across available cores, matching theoretical expectations for ideal parallel workloads.

C++ MPI implementation

Since MPI requires special profiling tools, and none of which are Mac supported, I ended up building a custom header file to profile my code and give a CSV output. It shows the different function calls, and how much time it takes in a particular run/recording of the code. The baseline metrics indicate the C++ code running serially, and the other numbers show the parallelisation in different number of processes in MPI

- Overview:
    - Employs MPI (Message Passing Interface) for process-level parallelism.
    - Each process handles different files—shown results for 2, 4, and 6 processes.
- Performance Results:

Table 6.

	Algo 1		Algo 2		Algo 3	
	Total Time	Core Logic Time	Total Time	Core Logic Time	Total Time	Core Logic Time
Baseline	1.00723s	0.0075448s	0.97622s	0.00933440s	1.03232s	0.034601s
2 process	0.456134s	total across all files = 0.00617613s, average per file = 6.17613e-05s	0.506421s	total across all files = 0.00981296s, average per file = 9.81296e-05s	0.571996s	total across all files = 0.0297835s, average per file = 0.000297835s
4 process	0.279698s	total across all files = 0.00640112s, average per file = 6.40112e-05s	0.261073s	total across all files = 0.00927867s, average per file = 9.27867e-05s	0.224399s	total across all files = 0.0246395s, average per file = 0.000246395s
6 process	0.218963s	total across all files = 0.00720792s, average per file = 7.20792e-05s	0.202146s	total across all files = 0.0104243s, average per file = 0.000104243s	0.165424s	total across all files = 0.0260887s, average per file = 0.000260887s

rank0-timings-85532

label	calls	total_ms	avg_ms
compute_from_vector_pdf_algo1	50	780.551	20.051029
get_words	100	294.865	2.948653

Fig 7: 2 process MPI implementation

rank0-timings-70369

label	calls	total_ms	avg_ms
compute_from_vector_pdf_algo1	25	506.450	20.258005
get_words	50	171.460	3.429204

Fig 8: 4 process MPI implementation

rank0-timings-86972

label	calls	total_ms	avg_ms
compute_from_vector_pdf_algo1	17	450.748	26.514577
get_words	34	149.524	4.397756

Fig 9: 6 process MPI implementation

The attached figures from different process configurations (2, 4, and 6) showcase the output from the custom profiling module, highlighting both the total and average execution time for the core logic. As these screenshots reveal, the time per function call and the overall computation time decrease consistently as parallelism increases, directly evidencing the benefits of effective workload distribution.

Additionally, the summary table further validates the embarrassingly parallel nature of the task by showing relatively constant core logic times per file, even as the total execution time decreases with more processes. This reflects nearly perfect scaling, with each individual process working independently on separate files. Collectively, these results confirm that my approach achieves ideal parallel speedup with minimal overhead or contention, and demonstrates the practical benefits of a truly embarrassingly parallel workload.

References:

1. David L. Tabb, Melissa R. Thompson, Gurusahai Khalsa-Moyers, Nathan C. VerBerkmoes, W. Hayes McDonald, MS2Grouper: Group Assessment and Synthetic Replacement of Duplicate Proteomic Tandem Mass Spectra, Journal of the American Society for Mass Spectrometry, Volume 16, Issue 8, 2005, Pages 1250-1261, ISSN 1044-0305, https://doi.org/10.1016/j.jasms.2005.04.010.
2. https://training.galaxyproject.org/training-material/topics/metabolomics/tutorials/gc_ms_with_xcms/tutorial.html#compute-similarity-scores
3. R. Pelikan and M. Hauskrecht, "Efficient Peak-Labeling Algorithms for Whole-Sample Mass Spectrometry Proteomics," in *IEEE/ACM Transactions on Computational Biology and Bioinformatics*, vol. 7, no. 1, pp. 126-137, Jan.-March 2010, doi: 10.1109/TCBB.2008.31
4. Lin A, Deatherage Kaiser BL, Hutchison JR, Bilmes JA, Noble WS. MS1Connect: a mass spectrometry run similarity measure. Bioinformatics. 2023 Feb 3;39(2):btad058. doi: 10.1093/bioinformatics/btad058. PMID: 36702456; PMCID: PMC9913042.
5. Lea G. Johnsen, Peter B. Skou, Bekzod Khakimov, Rasmus Bro, Gas chromatography – mass spectrometry data processing made easy, Journal of Chromatography A, Volume 1503,2017, Pages 57-64, ISSN 0021-9673, https://doi.org/10.1016/j.chroma.2017.04.052.
6. Chinnathambi, S. ., & Ganapathy, G. . (2024). Extraction of Features from the GC-MS Chromatogram Unstructured Data using Multi-Class and Multi-Label Classification for the Injection of Preprocessed Dataset into Machine Learning Algorithms applicable for E-Nose. International Journal of Intelligent Systems and Applications in Engineering, 12(3), 138–145.
7. Automating GC/MS Analysis for High-Quality Data. Lab Manager, 2023.https://www.labmanager.com/automating-gc-ms-analysis-for-high-quality-data-31283
8. W. Jiang, K. Li, and M. Chan, "Plot2Spectra: an automatic spectra extraction tool for digitizing published chromatograms," *Bioinformatics*, vol. 37, no. 4, pp. 560-567, 2021.
9. P. Du, W. Kibbe, and S. Lin, "Improved peak detection in mass spectrum by incorporating continuous wavelet transform-based pattern matching," *Bioinformatics*, vol. 22, no. 17, pp. 2059-2065, 2006.
10. J. Frank, J. Bandeira, and L. Smith, "PeakDetective: A semisupervised deep learning tool for peak validation and curation," *Nature Methods*, vol. 21, pp. 1245-1253, 2024
11. https://pymassspec.readthedocs.io/en/master/50_peak_alignment_by_dynamic_programming.html
12. T. H. Cormen, C. E. Leiserson, R. L. Rivest, and C. Stein, *Introduction to Algorithms*, 3rd ed., MIT Press, 2009.
13. M. Treen, P. Singer, and D. Monroe, "SIMILE: Optimal assignment of fragment ions across tandem mass spectra using bipartite graph matching," *Bioinformatics*, vol. 33, no. 13, pp. 1972-1980, 2017.
14. Dan Bylund, Rolf Danielsson, Gunnar Malmquist, Karin E. Markides, Chromatographic alignment by warping and dynamic programming as a pre-processing tool for PARAFAC modelling of liquid chromatography–mass spectrometry data, Journal of Chromatography A, Volume 961, Issue 2, 2002, Pages 237-244, ISSN 0021-9673, https://doi.org/10.1016/S0021-9673(02)00588-5.
15. Needleman SB, Wunsch CD. A general method applicable to the search for similarities in the amino acid sequence of two proteins. J Mol Biol. 1970 Mar;48(3):443-53. doi: 10.1016/0022-2836(70)90057-4. PMID: 5420325.

16. An Integrated Method for Spectrum Extraction and Compound Identification from GC/MS Data," published in the Journal of the American Society for Mass Spectrometry, 1999, Volume 10, pages 770-781
17. Smith CA, Want EJ, O'Maille G, Abagyan R, Siuzdak G. XCMS: processing mass spectrometry data for metabolite profiling using nonlinear peak alignment, matching, and identification. Anal Chem. 2006 Feb 1;78(3):779-87. doi: 10.1021/ac051437y. PMID: 16448051.